

Assignment

📌 Section A — Arithmetic with Gene Expression Counts

Q1. Add Two Gene Counts

Background: You sequenced Gene A and Gene B from a biopsy. Gene A had **120 reads**, and Gene B had **200 reads**. You want to calculate their combined expression.

Task:

Add the two counts using R.

Hint: use +.

Q2. Multiply Read Depths from Two Runs

Background: You received **500,000** and **600,000** reads from two sequencing runs. This might happen if you re-sequence a sample to improve coverage.

Task:

Multiply the two read counts to estimate total read pairs.

*Hint: use *.*

Q3. Calculate Fold-Change in Expression

Background: Gene X had **300 reads in Control** and **150 reads in Treated**. Fold-change helps determine whether a gene is upregulated or downregulated.

Task:

Calculate fold-change by dividing treated count by control.

Hint: use /.

Q4. Remainder of Reads per Lane

Background: Each sequencing lane holds **250,000** reads. Your sample had **1,001,000** reads.

Task:

Use the modulo operator to find leftover reads that didn't fit evenly.

Hint: use %%.

Q5. Store a Gene Count in a Variable

Background: You counted **350 reads** for a novel immune gene.

Task:

Assign this number to a variable named `gene_x_count`. Then print it.

Hint: use <-.

Q6. Add Counts from Two Patients

Background: Patient A had **100 reads**, and Patient B had **120 reads** for the same gene.

Task:

Store both counts in variables and calculate the total expression.

Hint: use variable names + +.

Q7. Update a Mistaken Value

Background: The count from Patient B was updated from **120** to **130** after data correction.

Task:

Update the value and recalculate the total.

Hint: reassign the variable.

Q8. Store the Name of a Marker Gene

Background: You're studying **CD274**, an immune checkpoint marker also known as PD-L1.

Task:

Store "CD274" in a variable called `marker_gene`.

Hint: use quotes " for text.

Section B — Working with Data Types

Q9. Check the Data Type of a Count

Background: R needs to know if your data is numeric before performing calculations.

Task:

Assign `x <- 100`, then check its data type with `class(x)`.

Q10. Check the Type of a Gene Name

Background: Text like "TP53" is stored as a *character* in R.

Task:

Assign "TP53" to a variable and check its class.

Q11. Logical Flag for Marker Gene

Background: Sometimes we want to tag a gene as a marker using TRUE/FALSE logic.

Task:

Create a variable `is_marker <- TRUE` and check its type using `class()`.

Section C — Vectors for Gene Expression Data

Q12. Create a Vector of Gene Counts

Background: You collected counts for **TP53**, **BRCA1**, and **EGFR**: 120, 150, and 130 respectively.

Task:

Create a vector `counts <- c(120, 150, 130)`.

Q13. Assign Gene Names to Counts

Background: You want each number in the vector to be clearly labeled with a gene name.

Task:

Assign names using `names(counts) <- c("TP53", "BRCA1", "EGFR")`.

Q14. Print the Named Vector

Background: You want to confirm that each count is correctly labeled.

Task:

Print the counts vector and inspect its names.

Q15. Extract One Gene's Count

Background: EGFR is of special interest due to its role in cancer.

Task:

Extract the count for "EGFR" using its name.

Q16. Extract the First Two Genes

Background: You are currently analyzing only **TP53** and **BRCA1**.

Task:

Extract these using position-based indexing (e.g., [1:2]).

Section D — Managing Sample and Gene Metadata

Q17. Create Sample Names

Background: You processed 3 samples from a patient.

Task:

Create a vector `samples <- c("Sample1", "Sample2", "Sample3")`.

Q18. Combine Gene Names with Counts

Background: You stored gene names separately and now want to link them to their counts.

Task:

Create:

```
genes <- c("TP53", "BRCA1", "EGFR")
```

```
counts <- c(120, 150, 130)
```

Then name counts using genes.

Q19. Check the Class of the Count Vector

Background: You need to ensure your counts are numeric for analysis.

Task:

Use `class(counts)` to check the type.

Section E — Data Conversion and Filtering

Q20. Create Patient IDs with Sequence

Background: You sequenced 5 patients. Each ID should follow the format "Patient_1" to "Patient_5".

Task:

Use `paste("Patient", 1:5, sep = "_")` to generate the list.

Q21. What Happens in a Mixed Vector?

Background: You mistakenly created a vector: `c("TP53", 100, TRUE)`.

R will try to make all elements the same type.

Task:

Use `class()` on this vector to see the result.

Q22. Convert a Character to Numeric

Background: A count was read as "150" (a character string). You need it as a number.

Task:

Convert it using `as.numeric("150")`.

Q23. Addition with Mixed Types (Fails)

Background: You try `x <- 5` and `y <- "6"`, then run `x + y`.

Task:

Try it and observe the error. Why does it fail?

Q24. Fix the Error and Add

Background: You realize you need to convert "6" to a number before adding.

Task:

Use `as.numeric(y)` and then add to x.

Q25. Filter Genes with High Expression

Background: You want to analyze only highly expressed genes (>100 reads).

Task:

From this vector:

```
counts <- c(TP53 = 120, BRCA1 = 90, EGFR = 310)
```

Extract genes with values greater than 100 using logical filtering.